

Do Caos a Organização – Versionamento com Git



Prof. Anderson Augusto Bosing

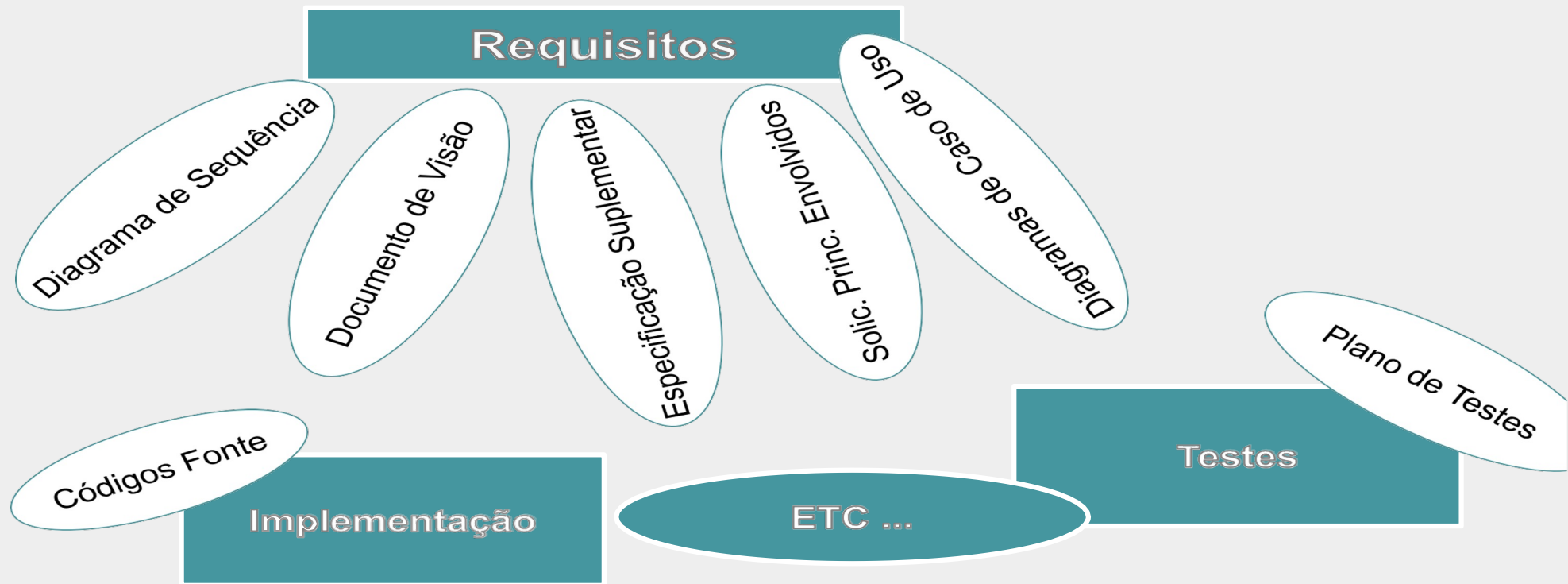
Controles e Estratégias de Versionamento com GIT

Uma característica das mais importantes do software é: Sempre sofre modificações.

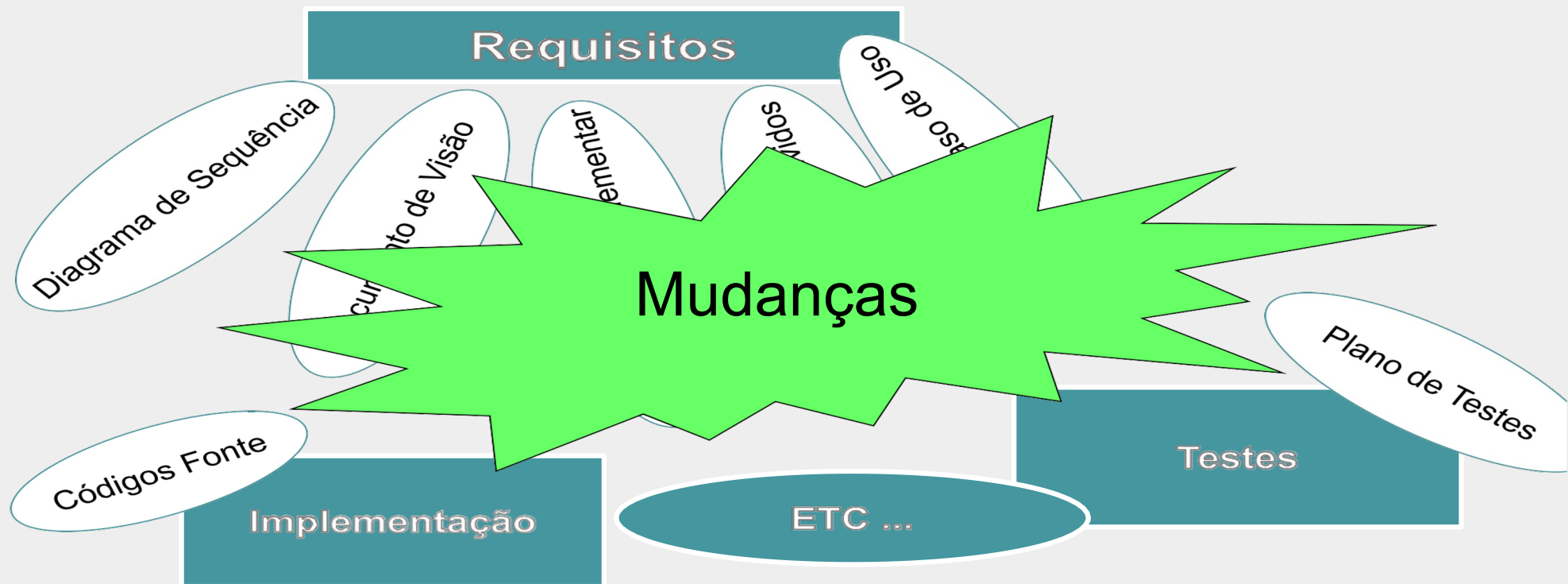
Quando o software deixa de ser modificado?



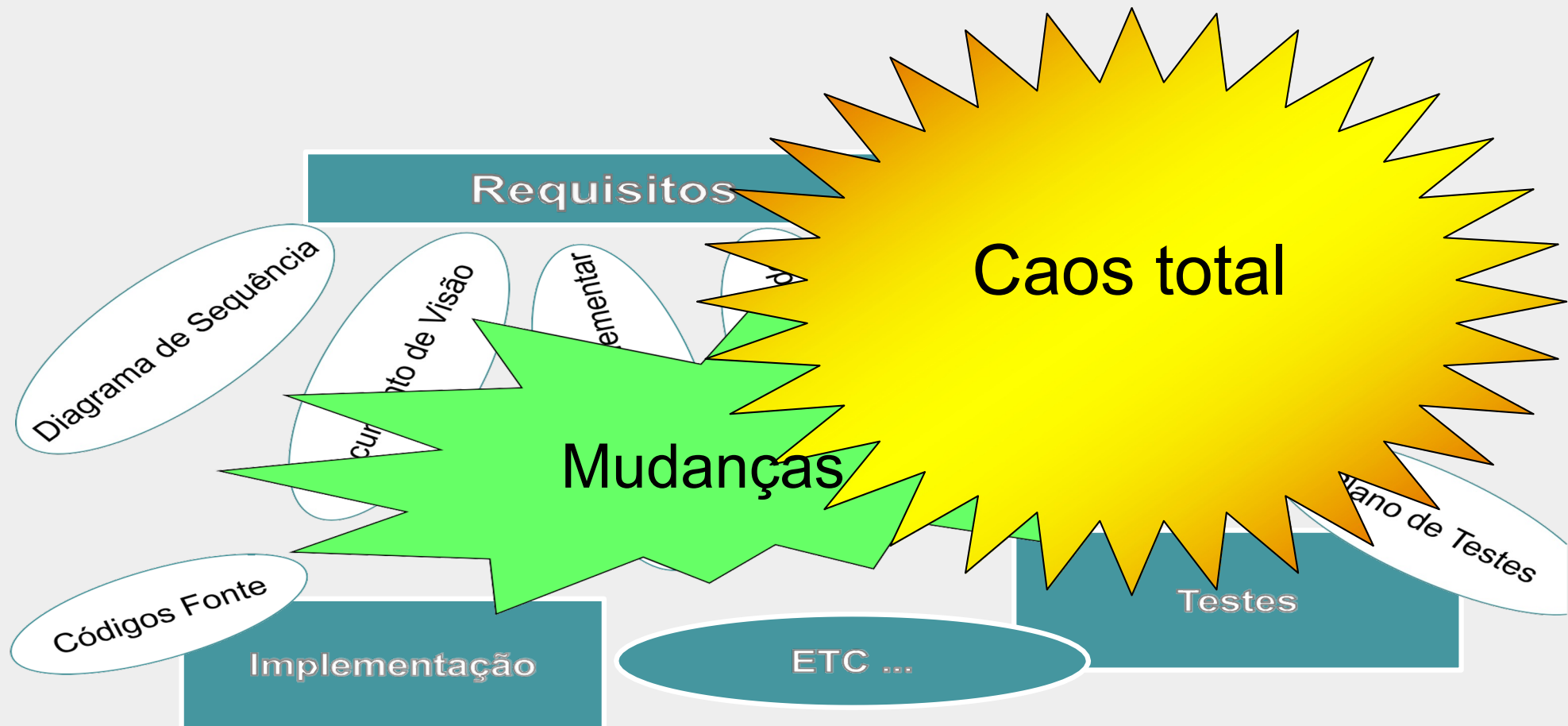
Controles e Estratégias de Versionamento com GIT



Controles e Estratégias de Versionamento com GIT



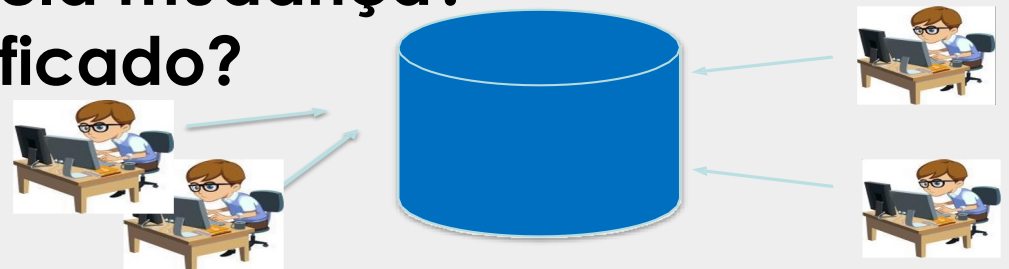
Controles e Estratégias de Versionamento com GIT



Motivação

- Falta de sincronismo entre as atividades.
- Notificação limitada.
- Conflito entre as atividades paralelas.
- Falta de controle de modificações.

Em que versão foi realizada a correção?
Qual é a versão mais atual?
Quem foi o responsável pela mudança?
O que realmente foi modificado?
Quando?



Sistemas de Controle de Versão

Os sistemas de controle de versão são ferramentas de software que ajudam as equipes de software a gerenciar as alterações ao código-fonte ao longo do tempo.

Como os ambientes de desenvolvimento aceleraram, os sistemas de controle de versão ajudam as equipes de software a trabalhar de forma mais rápida e inteligente.

Sistemas de Controle de Versão

Para quase todos os projetos de software, o código-fonte é como uma **mina de ouro**: um bem precioso com valor deve ser protegido.

Para a maioria das equipes de software, o código-fonte é um repositório de **conhecimento inestimável** do domínio do problema que os desenvolvedores coletaram e aprimoraram com muito cuidado.

Sistemas de Controle de Versão

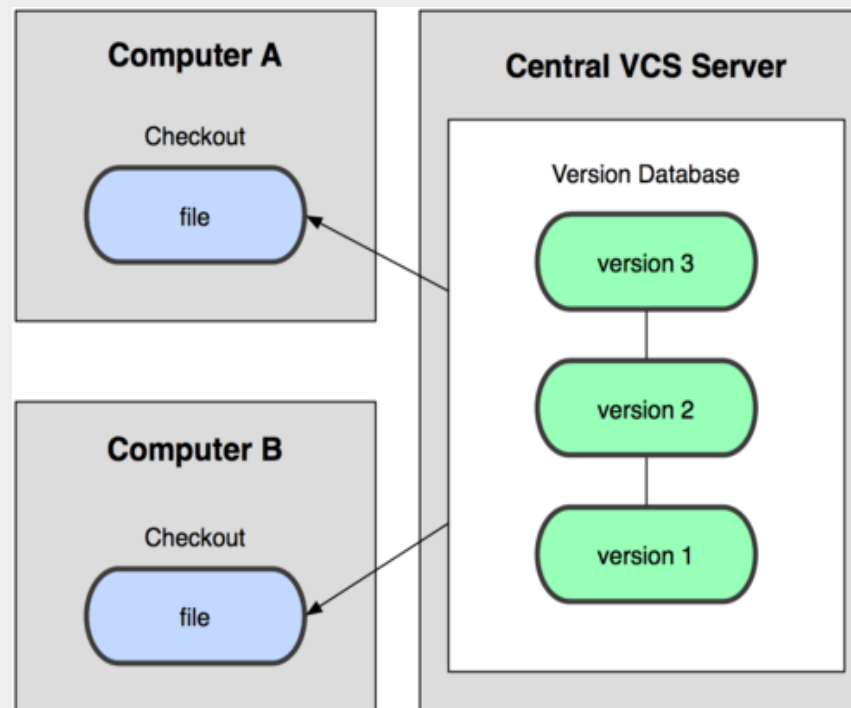
- Controlam o evolução do código do projeto.
- Possibilitam encontrar e reverter problemas ou alterações incluídos no código facilmente.
- O controle de versão protege o código-fonte contra catástrofes e a possível degradação causada por erro humano e consequências ruins.
- Sua eficácia é comprovada pela exigência em certificações como CMMI.

Benefícios

- Controle do histórico.
- Trabalho em equipe.
- Marcação e resgate de versões estáveis.
- Ramificação do projeto.

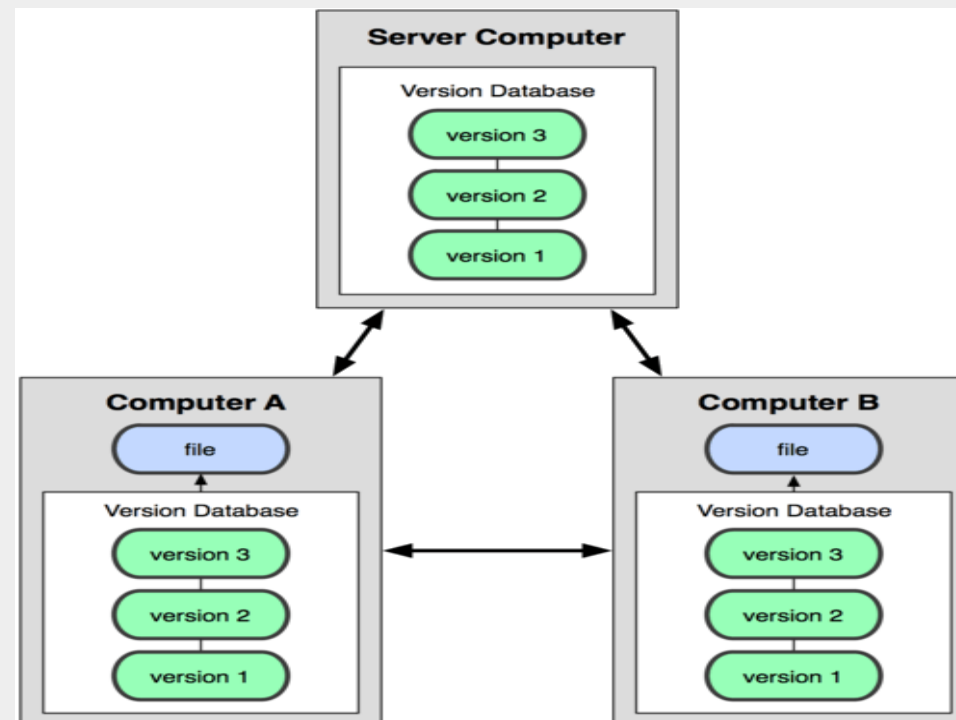
Controle de Versão com Git / Métodos de Controle de Versão

Centralizado



Controle de Versão com Git / Métodos de Controle de Versão

Distribuído



Sistemas de Controle de Versão

- CVS(1986) → SVN(2004) → Git(2005);
- Team Foundation Server(2005) / ClearCase(1992) (Comercial).

O Git

- Linus Torvalds;
- 2005;
- Controlar o código fonte do kernel Linux.



O Git - Critérios para o Desenvolvimento

- Não ser igual ao CVS.
- Suportar fluxo distribuído.
- Proteção contra corrompimento (acidente ou maldoso).
- Alta performance.

Pré-Requisitos

- Ter o Git instalado na máquina;
- Um repositório exclusivo para cada projeto.

Inicializando o repositório



~/repositorio: git init

Verificando o repositório



~/repositorio: git status

Rastreando o Arquivo



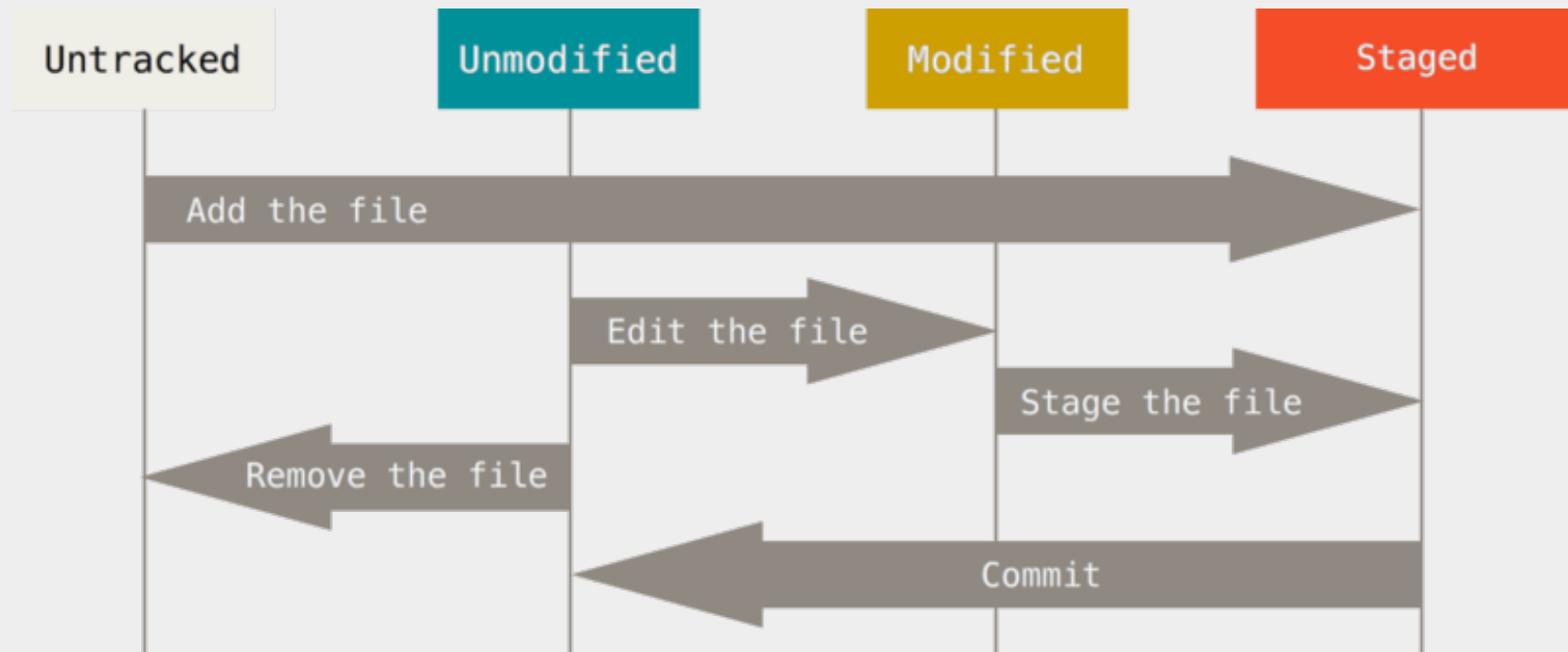
```
~/repositorio: git add [nome_arquivo]
```

Salvando o trabalho localmente



```
~/repositorio: git commit -m "[descricao]"
```

Estados dos Arquivos



Salvando o trabalho localmente



~/repositorio: git log

Salvando o trabalho localmente



~/repositorio: git diff

~/repositorio: git diff -- [file_name]

COLABORAÇÃO

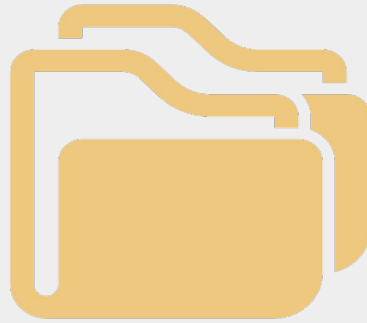


Prof. Anderson Augusto Bosing

Pré-Requisitos

- Um meio de transporte para o remoto (system file, HTTP, SSH);
- Uma chave de acesso ao remoto (quando necessário).

Clonando um repositório remoto



```
~/repositório_local: git clone [repo_system_url]
```

Enviando mudanças para o repositório



~/repositório_local: git push

Trazendo mudanças do repositório



~/repositório_local: git push

Merge – Verificando conflito



Unmerged paths:
(use "git add <file>..." to mark resolution)

both modified: bemvindo.txt

Merge – Interpretando conflito



Bem vindo ao mundo Git!

<<<<<< HEAD

alteracao?

=====

alteracao!

>>>>>> f8b208abf464823bd5f1eb6a4fcaabb9a26f8f7b

Merge – Após resolver conflito

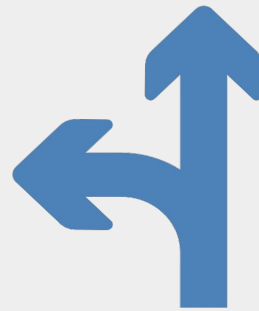


~/repositório_local: git add [nome_arquivo]

Ramificações

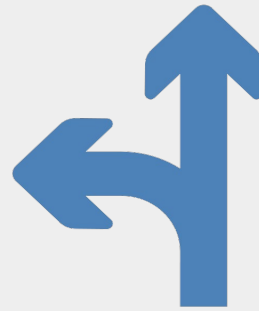
- Ramificações auxiliam no isolamento de diferentes trabalhos simultâneos;
- Ramos de desenvolvimento devem ser criados para garantir que o código fonte do trabalho atual tem como base uma versão estável e funcionando, podendo ser integrada de volta sem maiores problemas;
- Adiciona complexidade e necessidade de atenção por parte dos desenvolvedores;
- Garante que experiências possam ser feitas facilmente sem perder o trabalho que está incompleto em um ticket.

Manipulando ramificações



- ~/repositorio: `git checkout -b [identificador_branch]` / criar
- ~/repositorio: `git branch` / listar
- ~/repositorio: `git branch -d [identificador_branch]` / deletar / !!!
- ~/repositorio: `git checkout [identificador_branch]` / navegar

Manipulando ramificações



~/repositorio: `git merge [identificador_branch]`

Acabaram os comandos ?

Não.

Link para consulta de comandos.

https://training.github.com/downloads/pt_BR/github-git-cheat-sheet/



Arquivo .gitignore

O arquivo .gitignore é um arquivo de texto que diz ao Git quais arquivos ou pastas ele deve ignorar em um projeto.

Para criar um arquivo .gitignore, crie um arquivo de texto e dê a ele o nome de .gitignore (lembre-se de incluir o . no começo). Em seguida, edite esse arquivo conforme necessário. Cada nova linha deve listar um arquivo ou pasta adicional que você quer que o Git ignore.

Arquivo .gitignore

As entradas neste arquivo também podem seguir um padrão de correspondência.

***** é usado para corresponder a um caractere curinga .

/ é usado para ignorar nomes de caminhos relativos ao arquivo .gitignore .

é usado para adicionar comentários ao arquivo .gitignore.

Exemplo.gitignore para Java

Compiled class file
***.class**

Log file
***.log**

BlueJ files
***.ctxt**

Mobile Tools for Java (J2ME)
.mtj.tmp/

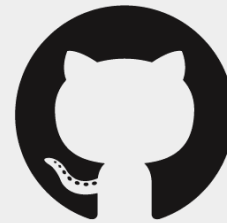
Package Files
***.jar**
***.war**
***.nar**
***.ear**
***.zip**
***.tar.gz**
***.rar**



Git e Github são a mesma coisa ?



git



GitHub

DEMONSTRAÇÃO PRÁTICA

- Criar conta no github.
- Criar repositório.
- Clonar repositório.
- Criar branches.
- Adicionar arquivos.
- Merge

VAMOS PRATICAR ?

Em caso de incêndio



- 1. git commit
- 2. git push
- 3. saia do prédio



Perguntas?



Prof. Anderson Augusto Bosing

Bibliografia Base

GIT - <https://git-scm.com/docs>



Prof. Anderson Augusto Bosing

Conclusão